# Trouvaille: AI-Powered Outfit Recommendation
# Deliverable 2 – Model Architecture & Evaluation

Group Members Romesha Afzaal 24L3001 Aliza Nadeem 24L-3024

## 1 Phase 1: Problem Definition

### 1.1 Problem Statement

Online fashion shopping is time-consuming and fragmented. A shopper preparing for an event, such as a wedding, concert, presentation or office day, must search for each clothing item separately, keep a budget in mind, and guess whether the pieces will look good together. Existing platforms rely on keyword search and category filters, which work poorly for requests like "a black outfit under Rs. 15,000 for a concert", because this single sentence combines colour, budget, occasion and multiple garments.

A second pain point is that most platforms ignore the individual. Recommendations are rarely adjusted for skin tone or body type, so shoppers often receive items that do not suit them, and rely on product ratings alone as a proxy for quality. Finally, no mainstream tool checks whether a complete outfit is appropriate for the type of event. The result is wasted time, over-budget purchases, returns, and low confidence in online buying.

Trouvaille addresses this by letting users describe what they need in plain language and returning a complete, budget-aware, occasion-appropriate outfit built from real products.

### 1.2 Target Audience

The primary users are online shoppers in Pakistan, particularly students and young professionals, who buy clothing for specific events on a limited budget and want quick, confident styling decisions. Secondary users include shoppers who find fashion choices overwhelming, and people who are underserved by generic recommendations because of their body type or skin tone.

From a research perspective, the project targets a gap between existing work: most recommender systems rank single items, whereas Trouvaille combines (i) natural-language query understanding, (ii) inclusive re-ranking by skin tone, body type and ratings, (iii) budget-constrained outfit composition, and (iv) an occasion-appropriateness check, all in one pipeline.

### 1.3 AI Justification

A traditional rule-based system is not suitable for this problem for three reasons.

**Free-form language understanding.** Users phrase requests in countless ways ("something black for a concert under 15k", "formal look for my cousin's wedding, size M"). Hand-written rules or regular expressions cannot cover this variety. A Large Language Model can reliably extract structured attributes such as budget, colour, occasion and size from unstructured text.

**Semantic product matching.** Keyword matching fails when a query and a product description use different words for the same idea (e.g. "edgy" or "smart casual"). Sentence embeddings represent queries and products as vectors, so products can be retrieved by meaning rather than exact wording, and the system generalizes to products it has never seen.

**Subjective, context-dependent judgment.** Deciding whether an outfit suits a wedding versus an office, or whether items complement each other, cannot be reduced to a fixed set of if–else rules because the criteria are subjective and overlap. Learned models capture these soft relationships from data and scale as the product catalogue grows, whereas a rule base would need constant manual maintenance.

Therefore, machine learning and deep learning are necessary for the language understanding, retrieval and compatibility components of Trouvaille, while deterministic logic is used only where exact constraints apply, such as enforcing the budget.

# 2 Phase 2: Data Acquisition & Preprocessing

## 2.1 Data Collection

Product data was collected from the online stores of ten Pakistani clothing brands (**Outfitters, Ethnc, Bonanza Satrangi, Lama, Lulusar, Faiza Saqlain, Hussain Rehar, Saira Shakira, J. (Junaid Jamshed) and Vanya**) using the script `scrape_products.py`. The script reads each store's public product feed (`/products.json`), checks the store's `robots.txt` before requesting data, waits between requests, retries failed requests, and records every step in a log file (`scrape_log.txt`), which serves as evidence of collection. For each product we store the title, type, tags, description, price, compare-at price, available sizes and colors, stock status, image URL and product URL. In total, **40,537** raw products were collected on **24 October 2026**. The number of raw products per brand was: J. 12,500; Bonanza Satrangi 10,089; Ethnc 4,960; Outfitters 3,885; Lulusar 2,781; Hussain Rehar 2,162; Lama 1,616; Faiza Saqlain 992; Vanya 834; and Saira Shakira 718. One additional store (Sapphire) was attempted but its feed returned a 404 error, so it was excluded. The scraper stops after a maximum of 50 pages per store, and this limit was reached for J., so its catalog may be larger than the 12,500 products collected.

## 2.2 Data Cleaning

The script `clean_data.py` performs the following steps (counts are in `cleaning_report.txt`):

- **Duplicates:** products repeated across pages or collections were removed using brand and product handle, and then brand, title and price (**3,403** removed).
- **Missing values:** products without a title or a valid price were dropped (**240**). Missing colors were filled by extracting the color from the title (remaining unknown colors are labeled `unknown`); missing sizes are labeled `UNKNOWN`; missing categories were inferred from the product type and title keywords, and products that could not be assigned a clothing category were dropped (**5,688**). The raw data had missing values mainly in `description` (7,401 products), `colors` (9,127), `compare_at_price` (7,277, meaning the product is not on sale), `sizes` (5,737) and `product_type` (838).
- **Standardization:** HTML was stripped from descriptions, sizes were mapped to a common scheme (e.g. `XXL` to `2XL`), and colors were normalized to a fixed vocabulary.

- **Scope:** children's products were removed because the application targets adult outfits (**4,978**).
- **Outliers:** extreme prices were removed using the interquartile-range rule on log-price within each category (**1,261** removed), which prevents data-entry errors and luxury items from distorting the budget-based search.

The final cleaned dataset contains **24,967** products (61.6% of the raw data) from **10** brands. However, the brand coverage is uneven: Faiza Saqlain and Saira Shakira keep only 4 and 3 products respectively, because their products carry first names as titles (e.g. "Seda") and have no category keyword, or have placeholder prices of zero, so they were removed by the category and price rules. In practice the dataset is built from eight brands.

### 2.3 Feature Strategy

The script `preprocess_features.py` prepares features for the later models.

- **Scaling:** price is min–max scaled to $[0, 1]$; log-price is standardized (z-score) because prices are strongly right-skewed; discount percentage is min–max scaled.
- **Encoding:** category, gender, grouped color (top 12 colors plus `other`), brand, color undertone, fit type and formality are one-hot encoded. Occasions and sizes can take several values per product, so they are multi-hot encoded.
- **Text features:** each product has a `text_for_embedding` field combining its title, attributes and description, to be encoded with sentence embeddings in the next phase.

**Derived labels for later components.** The Vibe Check Engine and the inclusive filter need information that stores do not provide, so we derived heuristic labels: (i) occasion labels (wedding, office, presentation, concert, casual) and a formality level from keyword rules; (ii) a fit type (fitted, relaxed, regular) and stretch flag from keywords; (iii) a color-undertone group (warm, cool, neutral) from a color-analysis rule, with the skin undertones each group is assumed to suit; and (iv) a plus-size indicator (2XL or larger offered) as a measurable proxy for body-type inclusivity. These labels are approximations and not ground truth, and will be validated and refined in later phases. Empty columns for manually assigned skin-tone and body-type labels are reserved for that purpose. Customer ratings are not available from the store feeds and are left for a later phase.

### 2.4 Exploratory Data Analysis

The catalog is dominated by sets (50.8% of products), followed by tops (20.4%) and bottoms (14.3%), while footwear (3.2%) and outerwear (2.1%) are rare; women's products make up 57.2% of the data, men's 22.9% and unisex 19.9%, and two brands, Bonanza Satrangi and J., supply 68.7% of all products. The occasion labels are multi-label and skewed toward casual (68.9%), and office (52.9%) and presentation (53.0%) have almost identical counts because their keyword lists overlap, so these two labels carry little independent information; wedding (42.2%) is probably inflated by broad keywords such as "embroidered" or "chiffon". Prices are strongly right-skewed (median PKR 4,000, mean 4,934, maximum 65,000, skewness 2.6), which justifies the log transform, and outerwear is the most expensive category (median PKR 7,693) while accessories and footwear are the cheapest. Price is moderately negatively correlated with discount percentage ($-0.54$), meaning cheaper items tend to be discounted more heavily, and 66% of products carry a discount, so the data reflects a sale period and the discount feature may not generalize to other times. Size inclusivity is limited:

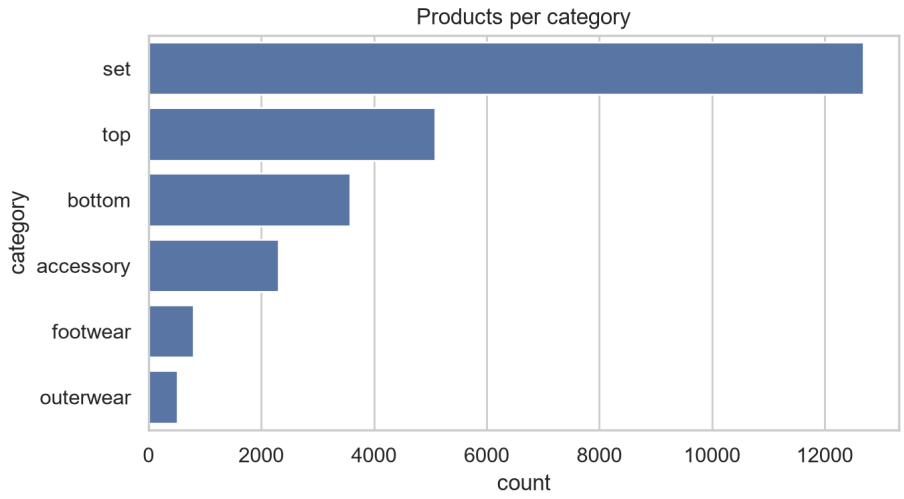



Figure 1: Number of products per category.

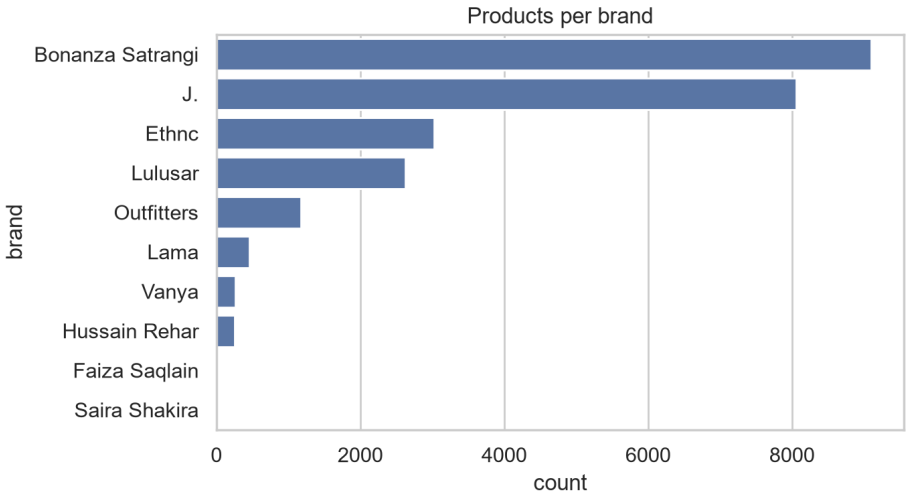



Figure 2: Number of products per brand.

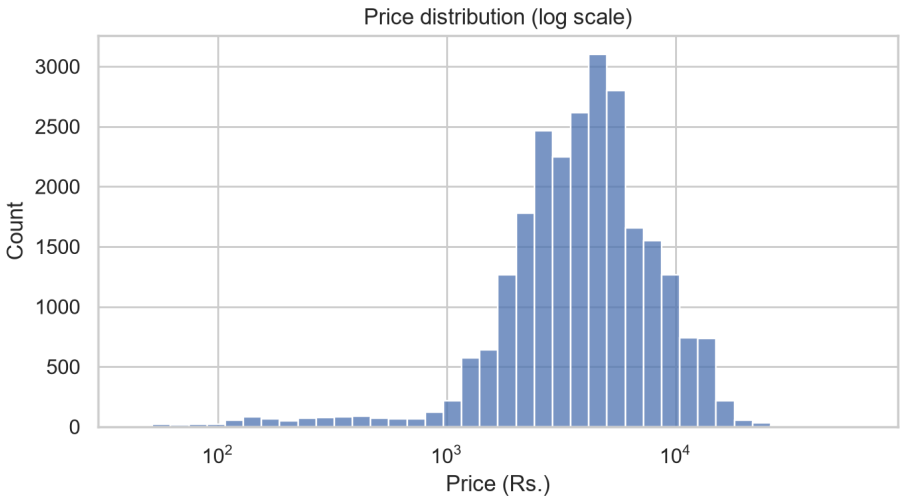



Figure 3: Price distribution (log scale).

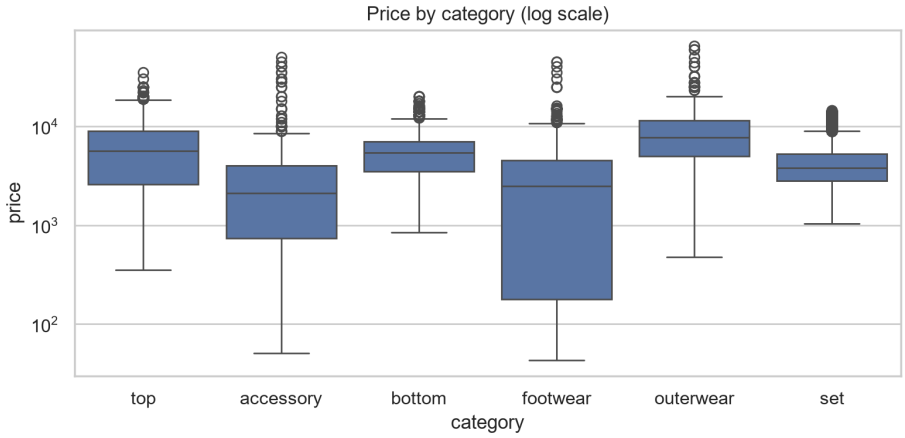



Figure 4: Price by category (log scale).

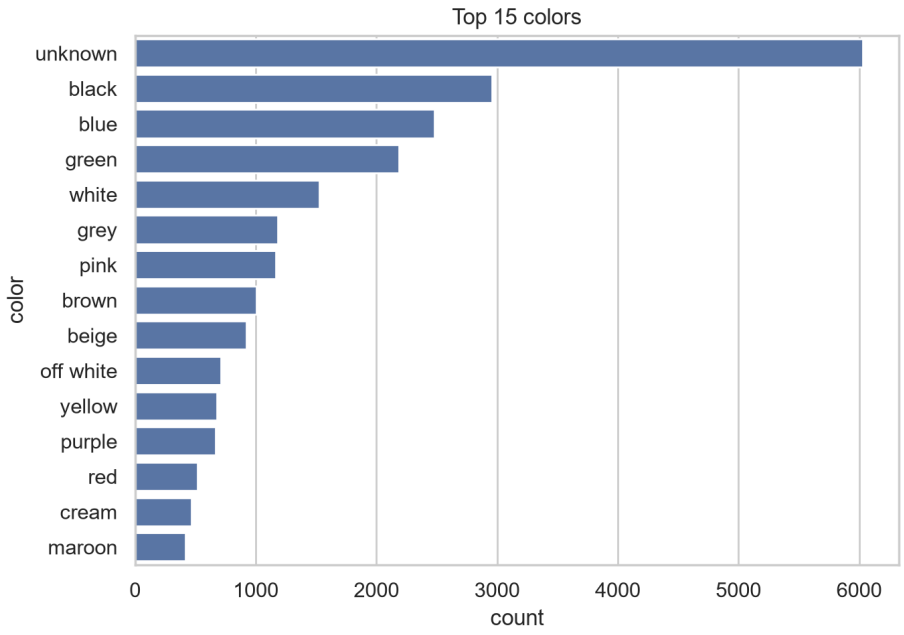



Figure 5: Most common colors.

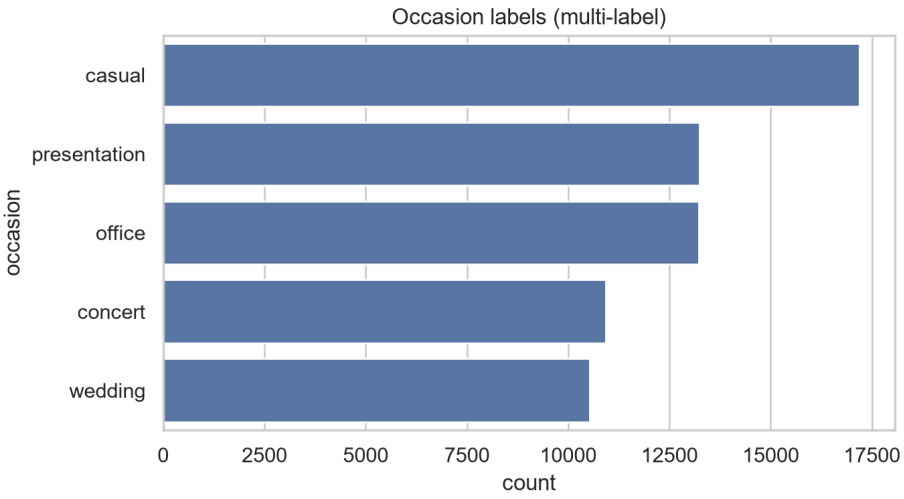



Figure 6: Distribution of heuristic occasion labels.

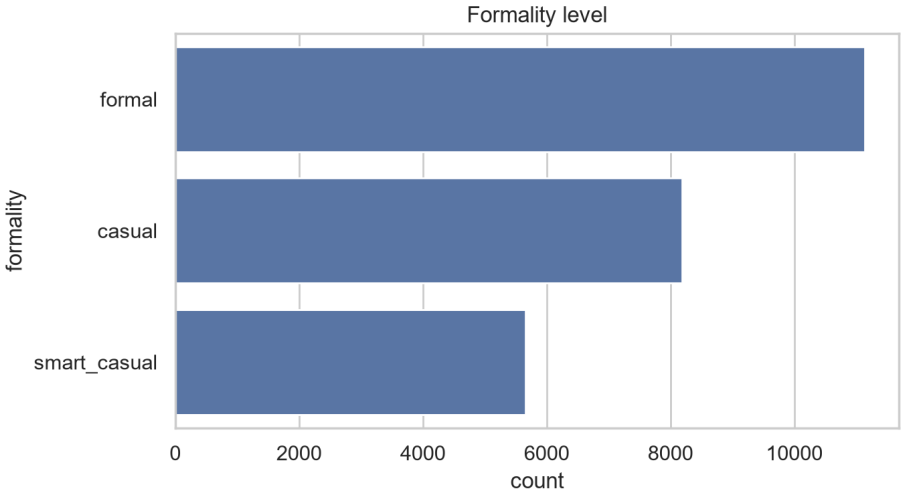



Figure 7: Distribution of formality levels.

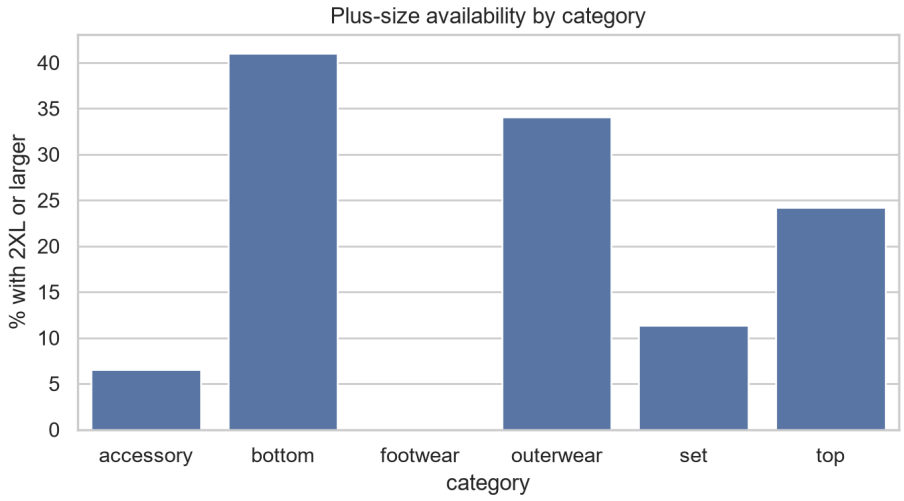



Figure 8: Share of products offering 2XL or larger, by category.

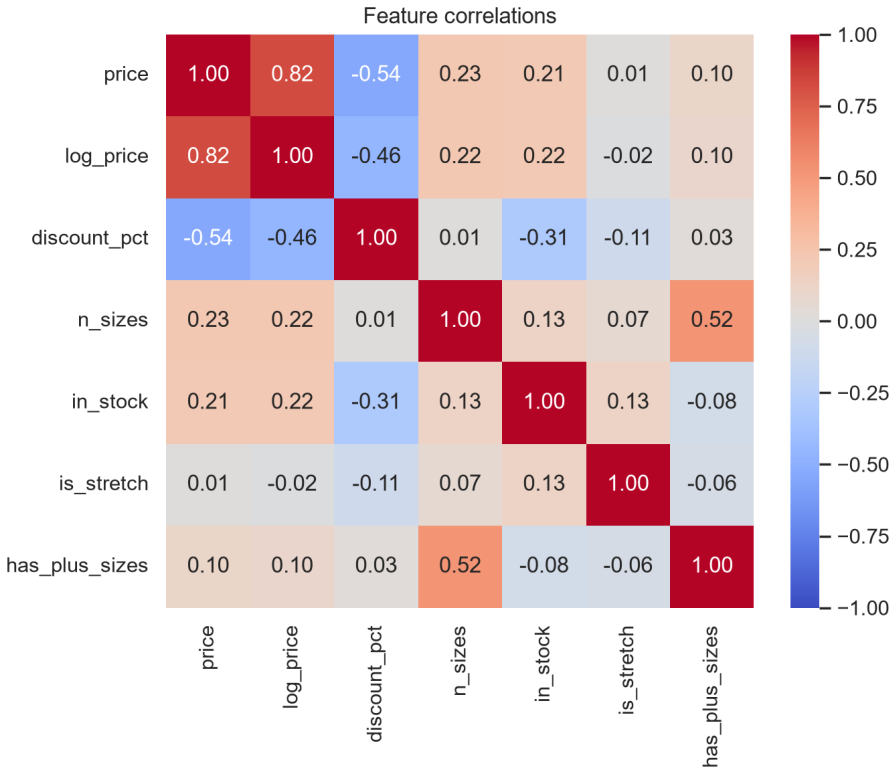



Figure 9: Correlation between numeric features.

only 17.9% of products (4,468) offer 2XL or larger, none of the footwear does, and about 91% of all plus-size products come from a single brand (J.), so the inclusive filter will have few options for most women's categories (14.4% of women's products offer plus sizes, against 42.2% of men's). For modeling, this means the data should be split in a way that is stratified by category and brand, the imbalance handled with class weights or resampling, the redundant features removed (`price` and `log_price` correlate at 0.82, and `has_plus_sizes` correlates at 0.52 with the number of sizes from which it is derived), and the heuristic labels evaluated carefully since they come from the same keywords as the text the models will see.

### 2.5 Limitations

Occasion, fit and undertone labels are keyword-based heuristics; ratings and skin-tone/body-type labels are not yet available; and the data covers only the selected brands at one point in time. In addition, the sample is concentrated in two brands, the luxury and bridal segment is almost absent after cleaning (Faiza Saqlain and Saira Shakira), the J. catalog was truncated at 50 pages, and 24.1% of products have an unknown color. Some prices, notably for Lama (median PKR 178), look unusually low compared with other brands and may use a different price unit in the feed; this should be checked before price is used as a model feature.

## 3 Phase 3: Model Architecture & Implementation

### 3.1 Task and Framework

The model implemented in this deliverable is the **occasion classifier** used by the Vibe Check Engine. Given a product's title and description, it predicts which of five occasions (wedding, office, presentation, concert, casual) the item is suitable for. A product can suit several occasions, so this is a **multi-label classification** problem. The solution is implemented in **PyTorch** (neural network), **Scikit-Learn** (baseline, splitting, metrics) and **Sentence-Transformers** (text encoder). All code is in `src/`.

### 3.2 Architecture

Each product text is first encoded by the pre-trained `all-MiniLM-L6-v2` sentence encoder into a 384-dimensional vector (the encoder is frozen; embeddings are computed once and cached). The vector is passed through a multilayer perceptron (MLP) with two hidden layers and one sigmoid output per occasion.

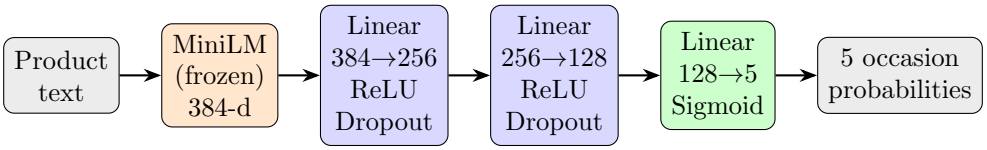



Figure 10: Architecture of the occasion classifier. Only the three linear layers are trained.

**Baseline.** To check that the neural network adds value, `baseline.py` trains a one-vs-rest **logistic regression** (`class_weight=balanced`) on the same embeddings.

| Component | Choice | Reason |
|---|---|---|
| Input | 384-d MiniLM sentence embedding | Captures meaning, not just keywords; small and fast on CPU |
| Hidden layers | 256 and 128 units, ReLU | Enough capacity for 5 labels with 17,476 training rows |
| Regularization | Dropout | Reduces co-adaptation; rate tuned (Table 2) |
| Output | 5 logits + sigmoid | Labels are not mutually exclusive, so independent probabilities are used instead of softmax |
| Loss | `BCEWithLogitsLoss` | Standard loss for multi-label targets; numerically stable |
| Optimizer | Adam | Adaptive step sizes; robust default for small MLPs |
| Threshold | 0.5 on each probability | Default; per-class tuning is future work |

Table 1: Design decisions.

**Preventing label leakage.** The occasion labels were derived from keyword rules applied to the same product text (Phase 2). The text field `text_for_embedding` contained the label string itself, so `make_text()` removes this prefix and only the title and description are given to the model. Keywords in the title can still hint at the label; this is discussed in the limitations.

### 3.3 Hyperparameter Tuning

We ran a full grid search over **learning rate** $\in \{10^{-2}, 10^{-3}, 10^{-4}\}$, **batch size** $\in \{64, 256\}$ and **dropout** $\in \{0.0, 0.3\}$, giving 12 configurations, each trained for 30 epochs with the Adam optimizer and seed 42. Configurations were ranked by macro F1 on the *validation* set only; the test set was not touched during tuning.

| LR | Batch | Dropout | Precision | Recall | F1 | AUC |
|---|---|---|---|---|---|---|
| **0.001** | **64** | **0.3** | **0.933** | **0.918** | **0.925** | **0.978** |
| 0.001 | 64 | 0 | 0.931 | 0.919 | 0.925 | 0.977 |
| 0.001 | 256 | 0.3 | 0.923 | 0.925 | 0.924 | 0.977 |
| 0.01 | 256 | 0 | 0.927 | 0.917 | 0.922 | 0.974 |
| 0.001 | 256 | 0 | 0.923 | 0.920 | 0.922 | 0.977 |
| 0.01 | 256 | 0.3 | 0.935 | 0.908 | 0.921 | 0.977 |
| 0.01 | 64 | 0.3 | 0.931 | 0.910 | 0.920 | 0.976 |
| 0.01 | 64 | 0 | 0.925 | 0.915 | 0.920 | 0.972 |
| 0.0001 | 64 | 0 | 0.906 | 0.893 | 0.899 | 0.963 |
| 0.0001 | 64 | 0.3 | 0.903 | 0.893 | 0.898 | 0.962 |
| 0.0001 | 256 | 0 | 0.884 | 0.883 | 0.883 | 0.948 |
| 0.0001 | 256 | 0.3 | 0.888 | 0.874 | 0.881 | 0.946 |

Table 2: Hyperparameter search results on the validation set (best row in bold).

**Final selection and justification.** The configuration with the highest validation macro F1 was selected: **learning rate 0.001, batch size 64, dropout 0.3**, with validation macro F1 0.925 and AUC-ROC 0.978. We selected on F1 rather than accuracy because F1 balances precision and recall across the five labels, and we used the validation set so that the test set stays an unbiased final check. The results show that the **learning rate** has by far the largest effect: averaged over

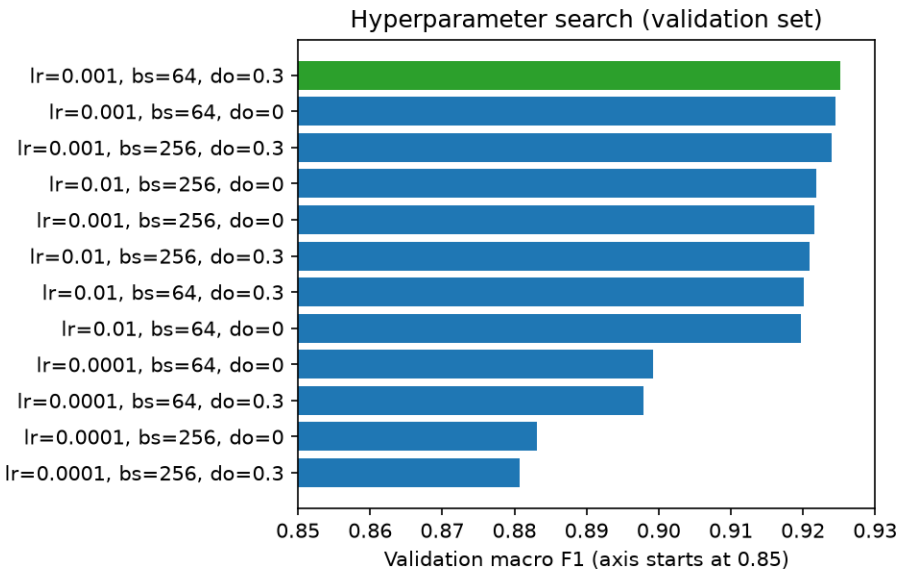



Figure 11: Validation macro F1 for each configuration; the selected one is green.

the other settings, $10^{-4}$ reached a validation F1 of only 0.890 because 30 epochs are too few for such small steps, while $10^{-3}$ (0.924) and $10^{-2}$ (0.921) were much better, and $10^{-3}$ scored slightly higher. A **batch size** of 64 was slightly better than 256 (0.915 versus 0.909), since smaller batches give more weight updates per epoch. **Dropout** made almost no difference (0.9117 without and 0.9115 with dropout on average), and the two best configurations differ by less than 0.001 F1. We kept dropout 0.3 because it was the top-ranked setting and, as Figure 12 shows, the network starts to overfit after about ten epochs, so a regularizer is a sensible safeguard even if its measured effect is small. Overall, the eight configurations with learning rate $10^{-3}$ or $10^{-2}$ lie within 0.006 F1 of each other, so the final result is not sensitive to the exact choice.

# 4 Phase 4: Training & Evaluation

## 4.1 Validation Strategy

The cleaned dataset (24,967 products) was split once into **70% train / 15% validation / 15% test** (17,476 / 3,745 / 3,746 products; `split_data.py`), **stratified by product category** with a fixed seed (42), so that rare categories such as footwear and outerwear appear in every split. The training set fits the weights, the validation set is used for hyperparameter selection, and the test set (3746 products) is used *once* for the final numbers. K-fold cross-validation was not used because the sentence embeddings are frozen and the dataset is large, so a single stratified hold-out gives stable estimates at a fraction of the cost.

## 4.2 Training Behaviour

Figure 12 shows the binary cross-entropy loss per epoch. Both curves fall quickly from about 0.30 during the first ten epochs. The validation loss then flattens at about 0.19 and even rises slightly after epoch 25, while the training loss keeps decreasing to about 0.075. The growing gap shows that the network begins to overfit after roughly ten epochs. Since the weights of the last epoch

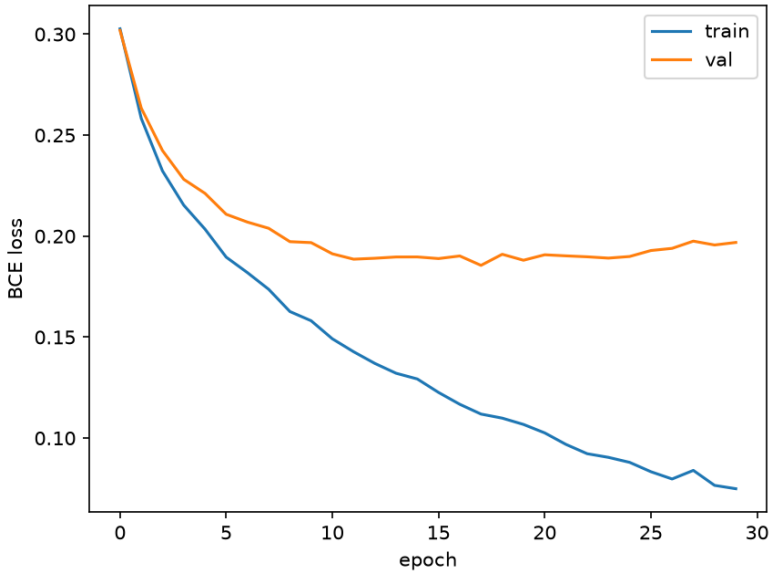



Figure 12: Training and validation BCE loss of the selected model.

are saved, early stopping (keeping the epoch with the lowest validation loss) would be a simple improvement; the gap is moderate and the test results below show that generalization is still good.

### 4.3 Metrics and Why They Fit This Problem

Plain accuracy is not informative for a multi-label task: each product has five yes/no decisions, and the labels are unevenly distributed (casual 68.7%, presentation 53.9%, office 53.8%, concert 43.1%, wedding 42.1% of the test set), so a model leaning on the frequent labels would still score well on accuracy alone. We therefore report:

- **Precision** – of the products the model assigns to an occasion, how many truly belong. Low precision means the outfit builder would recommend unsuitable items.
- **Recall** – of the products that suit an occasion, how many are found. Low recall shrinks the choice of items.
- **F1-score** – harmonic mean of the two; we use the *macro* average so rare occasions count as much as common ones.
- **AUC-ROC** – threshold-independent ranking quality of the probabilities, useful because the 0.5 threshold is not tuned.
- **Exact match and Hamming loss** – strict (all five labels right) and per-label error rates for the multi-label setting.

### 4.4 Results on the Test Set

The neural network clearly outperforms the logistic-regression baseline on the same validation data: macro F1 rises from 0.878 to 0.925 and AUC-ROC from 0.948 to 0.978. The gain comes from both

| Model | Precision | Recall | F1 | AUC-ROC |
|---|---|---|---|---|
| Logistic regression (validation) | 0.896 | 0.862 | 0.878 | 0.948 |
| MLP, selected (validation) | 0.933 | 0.918 | 0.925 | 0.978 |
| **MLP, selected (test)** | 0.933 | 0.923 | 0.928 | 0.980 |

Table 3: Macro-averaged metrics. Test micro-F1 is 0.928, Hamming loss 0.075 and exact-match ratio 0.751.

precision (0.896 against 0.933) and recall (0.862 against 0.918), so the non-linear hidden layers separate the occasions better than a linear model on the same embeddings. The test-set scores (F1 0.928, AUC-ROC 0.980) are about the same as the validation scores, so the model has not overfit the validation set through hyperparameter selection.

| Class | Support | Prevalence | Precision | Recall | F1 | AUC | FP | FN |
|---|---|---|---|---|---|---|---|---|
| casual | 2574 | 0.687 | 0.924 | 0.916 | 0.920 | 0.959 | 195 | 216 |
| concert | 1616 | 0.431 | 0.921 | 0.914 | 0.917 | 0.983 | 127 | 139 |
| office | 2014 | 0.538 | 0.940 | 0.938 | 0.939 | 0.985 | 120 | 125 |
| presentation | 2018 | 0.539 | 0.941 | 0.942 | 0.941 | 0.987 | 120 | 117 |
| wedding | 1577 | 0.421 | 0.940 | 0.906 | 0.923 | 0.983 | 91 | 148 |

Table 4: Per-class test results (FP/FN = false positives / false negatives).

### 4.5 Error Analysis

Overall the model reaches a macro F1 of 0.928 and an AUC-ROC of 0.980 on the test set, and 932 of the 3,746 test products (24.9%) have at least one wrong label. The five classes perform very similarly (Table 4): F1 ranges only from 0.917 (concert) to 0.941 (presentation). Errors are balanced between false positives and false negatives. *Casual* produces the most mistakes in absolute terms (195 false positives and 216 false negatives), partly because it is the most frequent label and has the lowest AUC (0.959); its definition is also the most vague, since it is the fallback label for products that match no other keyword rule. *Wedding* has the lowest recall (0.906, 148 false negatives), which means some wedding products are missed, while its precision is high (0.940).

*Office* and *presentation* obtain the best scores (F1 0.939 and 0.941), but this should not be read as a deeper understanding of formal wear. As noted in Phase 2, these two labels come from almost identical keyword lists, so they are nearly the same label, and in interactive tests the model predicted them together (for example, "formal blazer" scored 1.00 for both). The very high scores also reflect that the labels are derived by keyword rules from the same product text: the network mainly learns to reproduce those rules, so some of the remaining errors are caused by noisy labels (for example "blazer" was also predicted as casual with probability 0.90) rather than by the model.

Table 5 lists the most confident mistakes. Many involve products whose title contains a keyword that conflicts with the description, or labels produced by broad keyword rules (e.g. "embroidered" $\rightarrow$ wedding), which means part of the "error" is noise in the heuristic labels rather than a model failure.

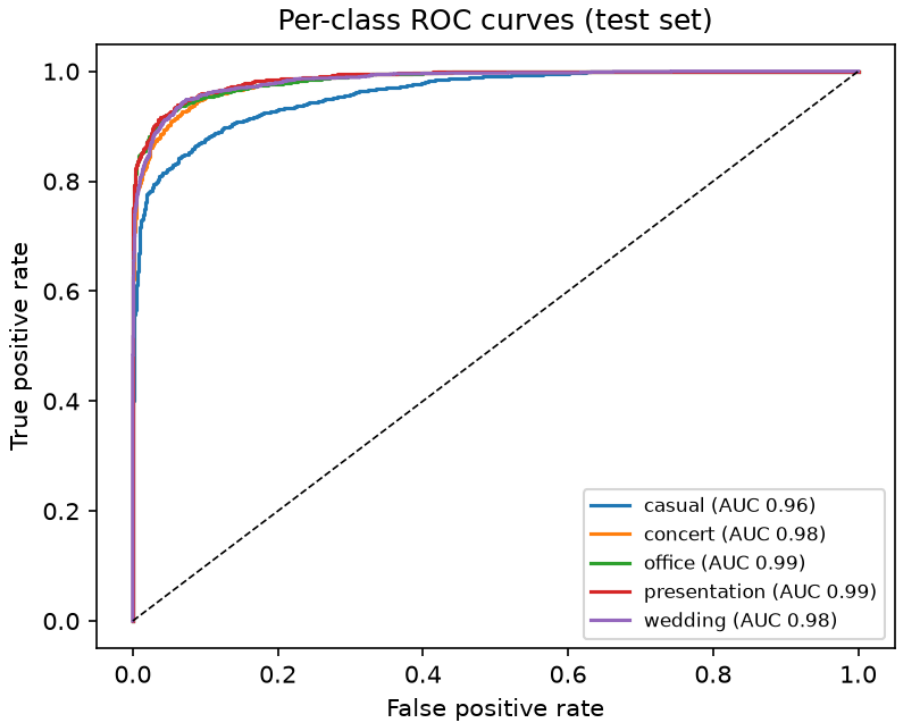



Figure 13: Per-class ROC curves on the test set.

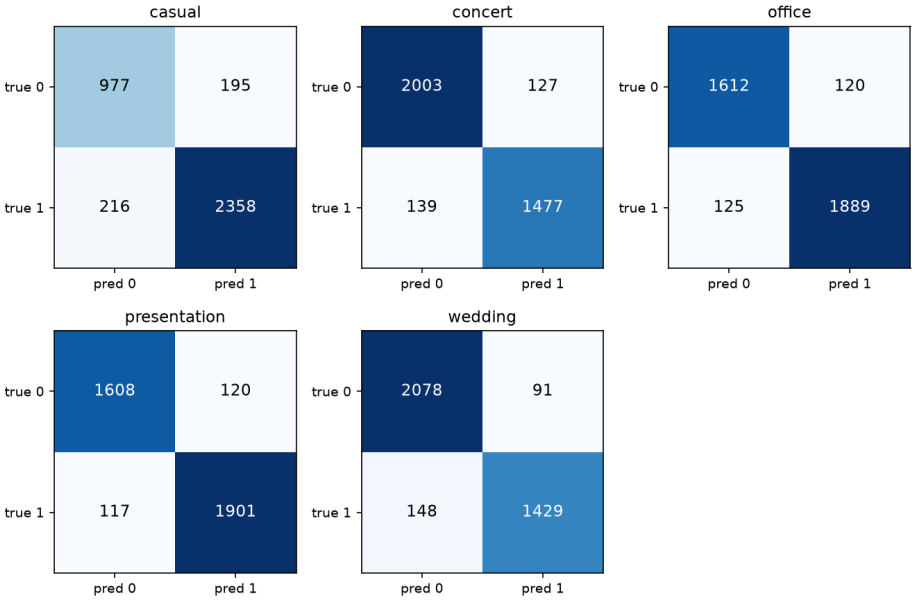



Figure 14: Binary confusion matrix of each occasion label on the test set.

| Product title | Brand | True labels | Predicted |
|---|---|---|---|
| EARRINGS (E2840/111/131) | Ethnc | concert | casual, office, presentation |
| CHUNKY SLIDES | Lama | concert | casual, concert |
| White Cotton Shalwar (MPS25SH1W004) | Bonanza Satr | casual, wedding | casual |
| Regil Long | Lulusar | casual | casual, concert, office, presentation |
| SHIRT (E0750/119/112) | Ethnc | wedding | office, presentation, wedding |
| SKIRT (E3602/108/901) | Ethnc | office, presentation | office, presentation, wedding |
| Bella - 3 PC (WSS25P3024) | Bonanza Satr | concert, office, presentation, wedding | casual, concert, office, presentation |
| BROWN TRENDY PESHAWARI SANDALS | J. | casual, wedding | casual |

Table 5: Most confident errors on the test set.

### 4.6 Bias Check

The project aims at inclusive recommendations, so we checked whether performance differs across groups. Skin-tone and body-type labels do not exist in the data yet (Phase 2), so **brand, gender, category and plus-size availability** are used as available proxies: if the model works much worse for a group, products from that group (e.g. women's wear, plus-size items, small brands) would receive worse recommendations.

Overall test macro F1 is 0.928. Performance is very similar for plus-size and other products (0.919 for products offering 2XL or larger, 0.930 for the rest, a gap of 0.011), so we found no meaningful disadvantage for plus-size items. Gender differences are small for men (0.950) and women (0.943), but *unisex* products score lower (0.879).

The largest differences appear across **brands** and **categories**. Among brands, F1 ranges from 0.971 (Outfitters) and 0.956 (Bonanza Satrangi) down to 0.867 (Ethnc), 0.765 (Lulusar) and 0.695 (Lama), a spread of 0.276. Lama has only 68 test products, so its estimate is noisy, but Lulusar (407 products) is a reliable signal that the model is clearly worse on this brand, 0.163 below the overall score. Among categories, sets score highest (0.957) while tops (0.855), accessories (0.860) and bottoms (0.878) are 5 to 7 points below the average. A likely reason is that the dataset is dominated by a few brands and by sets (50.8% of all products), so the model sees much more of their vocabulary and style of description, while other brands and single garments use different wording; we did not test this explanation directly. In the final application, recommendations for the weaker brands and categories (and for unisex items) would therefore be less reliable, so the occasion check should be treated with more caution for them. Possible remedies are oversampling weak groups, brand-balanced training and evaluation, and manual label checks on the weak subgroups.

**Limitations.** (i) The labels are keyword heuristics derived from the same text the model reads, so high scores partly measure how well the network reproduces our rules, not real shopper opinion; (ii) the data comes from two dominant brands (68.7% of products); (iii) no real demographic attributes are available, so the bias check cannot assess skin tone or body type directly; (iv) the threshold of 0.5 is the same for all labels; (v) the final epoch rather than the best epoch is saved. Future work:

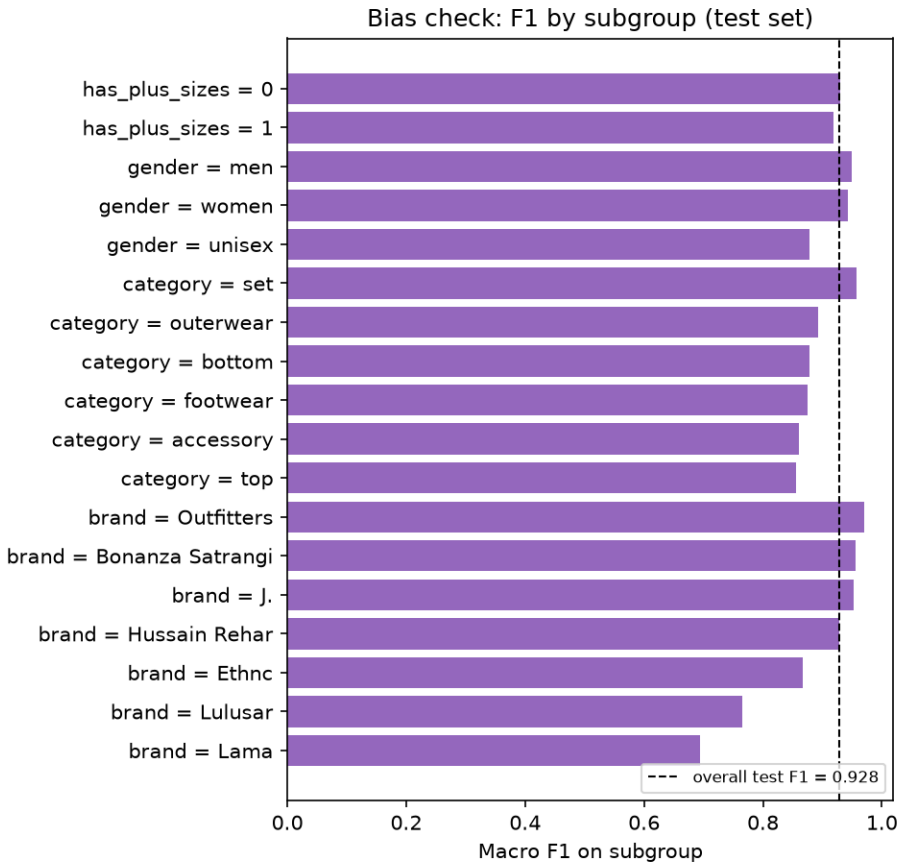



Figure 15: Macro F1 per subgroup on the test set (groups with fewer than 30 products are skipped).

| Attribute | Group | n | Macro F1 | Gap vs overall |
|---|---|---|---|---|
| brand | Lama | 68 | 0.695 | -0.233 |
| brand | Lulusar | 407 | 0.765 | -0.163 |
| brand | Ethnc | 446 | 0.867 | -0.061 |
| brand | Hussain Rehar | 36 | 0.928 | -0.000 |
| brand | J. | 1255 | 0.953 | 0.025 |
| brand | Bonanza Satrangi | 1356 | 0.956 | 0.028 |
| brand | Outfitters | 149 | 0.971 | 0.043 |
| category | top | 764 | 0.855 | -0.073 |
| category | accessory | 345 | 0.860 | -0.068 |
| category | footwear | 119 | 0.875 | -0.053 |
| category | bottom | 538 | 0.878 | -0.050 |
| category | outerwear | 77 | 0.893 | -0.035 |
| category | set | 1903 | 0.957 | 0.029 |
| gender | unisex | 771 | 0.879 | -0.049 |
| gender | women | 2130 | 0.943 | 0.015 |
| gender | men | 845 | 0.950 | 0.022 |
| has_plus_sizes | 1 | 677 | 0.919 | -0.009 |
| has_plus_sizes | 0 | 3069 | 0.930 | 0.002 |

Table 6: Subgroup macro F1 and gap to the overall test F1.

manual labels for a sample, per-class thresholds, early stopping, and fine-tuning the encoder.

## 5 Team Contributions & Version Control

All code was committed with the Git command line. The table below is generated automatically from the repository history by `src/contrib_table.py` and maps each commit author to the files (modules) they wrote.

**Model weights.** Trained weights (`mlp_occasion.pt`) are hosted on Google Drive; the link is in `README.md`. Code: `https://github.com/Romeshaafzaal/Trouvaille`.

| Author | Commit | Files / module | Message |
|---|---|---|---|
| Romesha Afzaal | `8b27a00` | src/split_data.py | Add 70/15/15 stratified split |
| Romesha Afzaal | `4dc676b` | src/baseline.py | Add embedding + logistic regression baseline |
| Romesha Afzaal | `2510b62` | src/mlp.py | Add PyTorch MLP with hyperparameter search |
| Romesha Afzaal | `a9eee91` | src/evaluate.py | Add test evaluation and bias check |
| Romesha Afzaal | `f2a6e5a` | requirements.txt | Update requirements for Phase 3 and 4 |
| Romesha Afzaal | `0f0b582` | README.md | Add model weights link to README |
| Aliza Nadeem | `f256f28` | src/predict.py | Add interactive predict script for occasion classification |
| Aliza Nadeem | `fe8240a` | src/report_utils.py | Add LaTeX table helper for report tables |
| Aliza Nadeem | `538a361` | src/error_analysis.py | Add per-class error analysis, ROC curves and confusion matrices |
| Aliza Nadeem | `cb984ae` | src/plot_results.py | Add hyperparameter and subgroup bias-check plots |
| Aliza Nadeem | `7a1edd1` | src/contrib_table.py | Add contribution table generator from git history |
| Aliza Nadeem | `559e3ec` | (none) | Merge pull request #1 from Romeshaafzaal/feature/error-analysis |
| Aliza Nadeem | `9c5bfaa` | (none) | Add Phase 4 figures: ROC, confusion matrices, tuning and bias check |
| Aliza Nadeem | `4c1b0f8` | report/tables/best_params.tex, report/tables/bias.tex, report/tables/errors.tex, report/tables/numbers.tex, report/tables/per_class.tex, report/tables/tuning.tex | Add generated metric and bias tables for the report |
| Aliza Nadeem | `05edd91` | report/contributions.tex, report/main.tex, report/phase3.tex, report/phase4.tex | Add Phase 3 and 4 report: architecture, tuning, evaluation, error analysis, bias check |
| Aliza Nadeem | `e73121a` | src/error_analysis.py, src/plot_results.py | Improve confusion matrix layout, error table format and tuning plot axis |
| Aliza Nadeem | `a676d10` | report/tables/errors.tex | Regenerate figures and error table with readable layout |
| Aliza Nadeem | `074c758` | (none) | Merge pull request #2 from Romeshaafzaal/feature/error-analysis |
| Aliza Nadeem | `a1102f4` | report/phase3.tex, report/tables/errors.tex, src/error_analysis.py | Fix table widths in design decisions and error examples |
| Aliza Nadeem | `2076127` | report/tables/errors.tex, src/error_analysis.py | Allow longer product titles in error examples table |
| Aliza Nadeem | `0056b16` | (none) | Merge pull request #3 from Romeshaafzaal/feature/error-analysis |
| Aliza Nadeem | `aac10c6` | report/tables/contributions.tex | Add contribution table generated from git history |
| Aliza Nadeem | `c14822f` | README.md | Add Deliverable 2 run instructions and results to README |

Table 7: Contribution table (Git commits for Deliverable 2).

| Member | Modules |
|---|---|
| Romesha Afzaal | `split_data.py` (70/15/15 split), `baseline.py` (embeddings + logistic regression), `mlp.py` (PyTorch MLP and grid search), `evaluate.py` (test metrics, subgroup bias check) |
| Aliza Nadeem | `error_analysis.py` (per-class metrics, ROC, confusion matrices, worst errors), `plot_results.py` (tuning and bias figures/tables), `predict.py` (interactive inference script), `report_utils.py`, `contrib_table.py`, and the LaTeX report for Phases 3 and 4 |

Table 8: Module ownership.